

Keresés és rendezés

Összehasonlítás alapú és kulcsmanipulációs algoritmusok

előadásvázlat

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések
- 4 Alsó becslés összehasonlítás alapú rendezésre
- 5 Összefésüléses rendezés
- 6 Gyorsrendezés
- 7 Kulcsmanipulációs rendezések
- 8 Összefoglalás

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések
- 4 Alsó becslés összehasonlítás alapú rendezésre
- 5 Összefésüléses rendezés
- 6 Gyorsrendezés
- 7 Kulcsmanipulációs rendezések
- 8 Összefoglalás

Legyen U egy halmaz, és $<$ egy kétváltozós reláció U -n. Ha $a, b \in U$ és $a < b$, akkor azt mondjuk, hogy a *kisebb, mint* b .

Definíció: rendezés

A $<$ reláció egy **rendezés**, ha teljesülnek a következők:

- ① $a \not< a$ minden $a \in U$ elemre ($<$ irreflexív);
- ② ha $a, b, c \in U$, $a < b$ és $b < c$, akkor $a < c$ ($<$ tranzitív);
- ③ tetszőleges $a \neq b \in U$ elemekre vagy $a < b$, vagy $b < a$ fennáll ($<$ teljes).

Ha $<$ egy rendezés U -n, akkor az $(U, <)$ párt **rendezett halmaznak** nevezzük.

- ▶ $\mathbb{Z}$ az egész számok halmaza. A $<$ rendezés a nagyság szerinti rendezés.
- ▶ Az abc betűinek Σ halmaza; a $<$ rendezést az abc-sorrend adja. Az x betű kisebb, mint az y betű, ha x előbb szerepel az abc-sorrendben, mint y .
- ▶ A Σ betűiből alkotott szavak Σ^* halmaza a **szótárszerű** (lexikografikus) rendezéssel.

Lexikografikus rendezés

Legyen $X = x_1x_2 \cdots x_k$ és $Y = y_1y_2 \cdots y_l$ két szó. Az X kisebb, mint Y , ha

- ▶ vagy $l > k$ és $x_i = y_i$ minden $i = 1, 2, \dots, k$ esetén;
- ▶ vagy pedig $x_j < y_j$ teljesül a legkisebb olyan j indexre, melyre $x_j \neq y_j$.

Tehát például $\text{kar} < \text{karika}$ és $\text{bor} < \text{bot}$.

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések
- 4 Alsó becslés összehasonlítás alapú rendezésre
- 5 Összefésüléses rendezés
- 6 Gyorsrendezés
- 7 Kulcsmanipulációs rendezések
- 8 Összefoglalás

Feladat

Adott az U halmaz véges $S = \{s_1, s_2, \dots, s_{n-1}, s_n\}$ részhalmaza és $s \in U$.

El akarjuk dönteni, hogy igaz-e $s \in S$, és ha igen, akkor melyik i -re teljesül $s_i = s$.

Hány összehasonlítás kell?

- ▶ Itt egy összehasonlítás: *Igaz-e, hogy $s_i = s$?*
Válasz: igen vagy nem.
- ▶ Legrosszabb esetben minden elemet végig kell nézni $\implies n$ összehasonlítás kell.
- ▶ Átlagosan $n/2$ összehasonlítás kell.

Rendezetlen halmazban tehát nem tudunk jobbat: minden elemet meg kell néznünk, amíg meg nem találjuk.

Barkochba

Gondolok egy számot 1 és 100 között — hány eldöntendő kérdésből lehet kitalálni?

Feladat

Adott az $(U, <)$ rendezett halmaz véges $S = \{s_1 < s_2 < \dots < s_{n-1} < s_n\}$ részhalmaza és $s \in U$. Összehasonlításokkal akarjuk eldönteni, hogy igaz-e $s \in S$, és ha igen, melyik i -re teljesül $s_i = s$.

- ▶ Itt egy összehasonlítás: *Mi a viszonya s -nek és s_i -nek?*
Válasz: $s_i = s$ vagy $s_i < s$ vagy $s_i > s$.
- ▶ Most **többet tudunk meg** egy kérdésből — ezt kell kihasználni.

Lineáris keresés

Sorban mindegyik elemmel összehasonlítjuk.

- ▶ Költség a legrosszabb esetben: n , mert lehet, hogy pont az utolsó volt.
- ▶ Költség átlagos esetben: $(n/2) + 1$.

Ez nem használja ki a rendezettséget — ugyanannyi, mint a rendezetlen esetben.

Tudunk-e lényegesen gyorsabbat?

Oszd meg és uralkodj!

Először a **középső** s_i -vel hasonlítunk. Ezután hasonló feladatot kapunk egy S_1 halmazra, amire viszont $|S_1| \leq |S|/2$. És így tovább:

$$|S_2| \leq \frac{|S|}{4}, \quad |S_3| \leq \frac{|S|}{2^3}, \quad \dots \quad |S_k| \leq \frac{|S|}{2^k}.$$

Példa: keressük meg, benne van-e a 21 az alábbi sorozatban!

15, 22, 25, 37, 48, 56, 70, 82

15, 22, 25, 37, 48, 56, 70, 82

15, 22, 25, 37, 48, 56, 70, 82

15, 22, 25, 37, 48, 56, 70, 82

Négy összehasonlítás után kiderül: a 21 **nincs benne**.

Addig kell csinálni, amíg $|S_k| = 1$ lesz. Innen

$$1 = |S_k| \leq \frac{n}{2^k} \implies 2^k \leq n \implies k \leq \lfloor \log_2 n \rfloor.$$

Ez $k + 1$ összehasonlítás volt, tehát

$$k + 1 \leq \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n + 1) \rceil.$$

Tétel

Ez optimális: nincs olyan kereső algoritmus, ami minden esetben kevesebb, mint $\lceil \log_2(n + 1) \rceil$ kérdést használ.

- ▶ $n = 100$ esetén $\lceil \log_2 101 \rceil = 7$ kérdés elég (és kell) — ez a barkochba válasza.
- ▶ $n = 10^6$ esetén is csak 20 kérdés!

Az alsó becslés bizonyítása — ellenség-stratégia

Az ötlet

Az **ellenség** nem is gondol egy számra, csak mindig úgy válaszol, hogy minél többet kelljen kérdezni.

- ▶ Ha egy kérdést felteszek, és az *igen* válasz után mondjuk x lehetőség jön szóba, akkor a *nem* esetén még $n - x - 1$ lehetőség marad. (A „ -1 ” az $s = s_i$ válasz miatt van.)
- ▶ Az ellenség úgy válaszol, hogy minél több lehetőség maradjon, így el tudja érni, hogy legalább $\frac{n-1}{2}$ marad.
- ▶ Két kérdés után legalább $\frac{\frac{n-1}{2} - 1}{2} = \frac{n}{2^2} - \frac{1}{2} - \frac{1}{2^2}$ marad.
- ▶ k kérdés után is marad még $\frac{n}{2^k} - \frac{1}{2} - \dots - \frac{1}{2^k}$ lehetőség.

Tehát teljesülnie kell, hogy $\frac{n}{2^k} - \frac{1}{2} - \dots - \frac{1}{2^k} \leq 1$, azaz

$$n \leq 2^k + 2^{k-1} + \dots + 1 = 2^{k+1} - 1 \implies \lceil \log_2(n+1) \rceil - 1 \leq k.$$

Ha még van egy lehetséges elem, akkor kell még $+1$ kérdés.



Feladat

Adott az $(U, <)$ rendezett halmaz véges $S = \{s_1, s_2, \dots, s_{n-1}, s_n\}$ részhalmaza.

Összehasonlításokkal keressük meg S minimális elemét, azaz egy olyan s_i elemet, hogy minden $i \neq j$ esetén $s_i < s_j$.

Hány összehasonlítás kell a legrosszabb esetben?

Feladat

Adott az $(U, <)$ rendezett halmaz véges $S = \{s_1, s_2, \dots, s_{n-1}, s_n\}$ részhalmaza.

Összehasonlításokkal keressük meg S minimális elemét, azaz egy olyan s_i elemet, hogy minden $i \neq j$ esetén $s_i < s_j$.

Hány összehasonlítás kell a legrosszabb esetben?

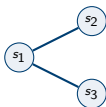
Tétel

n elem közül a minimális kiválasztásához legrosszabb esetben pontosan $n - 1$ összehasonlítás kell.

Két irányt kell belátni: $n - 1$ mindig *elég*, és $n - 1$ -nél kevesebb *nem mindig elég*.

$n - 1$ összehasonlítás mindig elég. Rendezzünk **kiesés versenyt**: mindig a kisebb elemet megtartva egy-egy összehasonlítás után. Mivel „mindenki pontosan egyszer kap ki, a győztest kivéve”, ez $n - 1$ összehasonlítást igényel.

$n - 1$ -nél kevesebb nem mindig elég. Legyenek az elemek egy gráf pontjai; ha kettőt összehasonlítottunk, húzzunk közöttük élet.



1. komponens



2. komponens

- ▶ Amíg a gráf **nem összefüggő**, bármely komponensében lehet a minimális elem — nem tudunk dönteni.
- ▶ Ha a gráf már összefüggő, akkor legalább $n - 1$ éle van, tehát kell ennyi összehasonlítás. $\square$

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések**
- 4 Alsó becslés összehasonlítás alapú rendezésre
- 5 Összefésüléses rendezés
- 6 Gyorsrendezés
- 7 Kulcsmanipulációs rendezések
- 8 Összefoglalás

Feladat

Adott az $(U, <)$ rendezett halmaz véges $S = \{s_1, s_2, \dots, s_{n-1}, s_n\}$ részhalmaza. Összehasonlításokkal rendezzük S elemeit növekvő sorrendbe, azaz keressünk olyan σ permutációt, hogy

$$s_{\sigma(1)} < s_{\sigma(2)} < \dots < s_{\sigma(n)}.$$

- ▶ **Input:** tömb, láncolt lista (vagy bármi).
- ▶ **Output:** általában olyan, mint az input.
- ▶ **Lépések:** elemek mozgatása, cseréje, összehasonlítása.

A rendezés önmagában is előforduló feladat, de előjön mint hasznos **adatszerkezet** is: rendezett halmazban könnyebb keresni (pl. telefonkönyv).

A vizsgálandó kérdések

- ▶ Hány **összehasonlítás** kell a legrosszabb esetben?
 - ▶ Hány összehasonlítás kell **átlagos** esetben?
 - ▶ Hány **csere** (mozgatás) kell a legrosszabb esetben?
 - ▶ Mennyi **plusz tárhely** szükséges?
-
- ▶ Egy algoritmus lehet kevés összehasonlítású, de sok mozgatású (és fordítva).
 - ▶ *Helyben* (in situ) rendez, ha konstans plusz tárhely elég.
 - ▶ *Konzervatív* (stabil), ha az egyenlő kulcsú elemek sorrendjét megőrzi.

Input: $A[1 : n]$ (rendezetlen) tömb. Ha valamely i -re $A[i] > A[i + 1]$, akkor a két cella tartalmát kicseréljük. A tömb elejéről indulva, közben cserélgetve eljutunk a tömb végéig. Ekkor a **legnagyobb elem $A[n]$ -ben van**. Ismételjük ezt az $A[1 : n - 1]$ tömbre, majd az $A[1 : n - 2]$ tömbre, stb.

```
procedure buborék
(* az A[1:n] tömböt növekvően (nem csökkenően) rendezi *)
for (j = n-1, j > 0, j := j-1) do
  for (i = 1, i <= j, i := i+1) do
    { ha A[i+1] < A[i], akkor cseréljük ki őket }
```

- ▶ összehasonlítások száma: $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$
- ▶ cserék száma: $\leq \frac{n(n - 1)}{2}$
- ▶ plusz tárhely: konstans (helyben rendez, konzervatív)

kiindulás	5	2	4	1	3
1. menet után	2	4	1	3	5
2. menet után	2	1	3	4	5
3. menet után	1	2	3	4	5
4. menet után	1	2	3	4	5

Minden menet végén a soron következő legnagyobb elem a helyére „buborékol”.

- ▶ Egyszerű, de $\mathcal{O}(n^2)$ — nagy n -re használhatatlan.
- ▶ Előnye: majdnem rendezett tömbön (korai kilépéssel) gyors lehet.

Alapötlet: a kártyalapok kézbe rendezése. Tegyük fel, hogy $A[1 : i - 1]$ már rendezett; szűrjük be $A[i]$ -t a helyére.

```
procedure beszűrásos rendezés
for (i = 2, i <= n, i := i+1) do
  x := A[i]; j := i-1
  while (j > 0 és A[j] > x) do
    A[j+1] := A[j]; j := j-1
  A[j+1] := x
```

- ▶ Legrosszabb eset (fordítva rendezett input): $\frac{n(n-1)}{2}$ összehasonlítás és mozgatás.
- ▶ Legjobb eset (már rendezett input): $n - 1$ összehasonlítás $\Rightarrow$ lineáris.
- ▶ Helyben rendez, konzervatív, kis n -re a gyakorlatban nagyon gyors — ezért használják a gyorsrendezés „rekurzió-aljaként”.

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések
- 4 Alsó becslés összehasonlítás alapú rendezésre**
- 5 Összefésüléses rendezés
- 6 Gyorsrendezés
- 7 Kulcsmanipulációs rendezések
- 8 Összefoglalás

Alsó becslés összehasonlítás alapú rendezésre

Ugyanaz, mintha barkochbában kellene kitalálni, hogy az elemek **melyik sorrendje** (**permutációja**) az igazi.

- ▶ Kezdetben $n!$ lehetséges sorrend jön szóba.
- ▶ Két elemet összehasonlítva a válasz **két részre** osztja a sorrendeket: ha pl. azt kapjuk, hogy $x < y$, akkor azok a sorrendek, amelyekben x hátrébb van y -nál, már nem jönnek szóba.
- ▶ Ha az ellenség megint úgy válaszol, hogy minél több sorrend maradjon meg, akkor k kérdés után még szóba jön $\frac{n!}{2^k}$ sorrend.
- ▶ Ha $\frac{n!}{2^k} > 1$, nem tudjuk megadni a rendezést. $\implies$

Tétel

Minden összehasonlítás alapú rendező módszer n elem rendezésekor legrosszabb esetben legalább $\log_2(n!)$ összehasonlítást használ.

Az alsó becslés nagyságrendje

A Stirling-formula szerint $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$, ezért

$$\log_2(n!) = n \log_2 n - n \log_2 e + \mathcal{O}(\log n) \approx n \log_2 n - 1.44 n.$$

Következmény

Összehasonlítás alapú rendezéssel $\Omega(n \log n)$ összehasonlítás alá nem lehet menni.

n	$\log_2(n!)$	$n^2/2$
10	22	50
100	525	5 000
1000	8 530	500 000

A négyzetes módszerek tehát **messze** nem optimálisak — keressünk $\mathcal{O}(n \log n)$ -es algoritmust!

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések
- 4 Alsó becslés összehasonlítás alapú rendezésre
- 5 Összefésüléses rendezés**
- 6 Gyorsrendezés
- 7 Kulcsmanipulációs rendezések
- 8 Összefoglalás

Összefésülés

Két már rendezett sorozat (tömb, lista, stb.) tartalmának **egy** sorozatba való rendezése:

$A[1 : k]$ és $B[1 : l]$ rendezett tömbök $\longrightarrow C[1 : k + l]$ rendezett tömb

- ▶ Nyilván $C[1] = \min\{A[1], B[1]\}$, pl. $A[1]$; ezt rakjuk át C -be és töröljük A -ból.
- ▶ Ezután $C[2] = \min\{A[2], B[1]\}$, stb.
- ▶ Minden összehasonlítás után legalább egy elem **végleg** a helyére kerül.

összehasonlítások száma: $k + l - 1$

<i>A</i>	<i>B</i>	<i>C</i>
12, 15, 20, 31	13, 16, 18	
15, 20, 31	13, 16, 18	12
15, 20, 31	16, 18	12, 13
20, 31	16, 18	12, 13, 15
20, 31	18	12, 13, 15, 16
20, 31	—	12, 13, 15, 16, 18
31	—	12, 13, 15, 16, 18, 20
—	—	12, 13, 15, 16, 18, 20, 31

Itt $k = 4$, $l = 3$, tehát legfeljebb $k + l - 1 = 6$ összehasonlítás kellett.

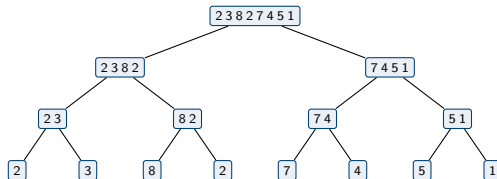
Összefésüléses rendezés (MERGE SORT)

Alapötlet

Rendezzük külön a tömb első felét, majd a második felét, végül fésüljük össze. Ezt csináljuk **rekurzívan**.

$$\text{MSORT}(A[1 : n]) := \text{MERGE}(\text{MSORT}(A[1 : \lceil n/2 \rceil]), \text{MSORT}(A[\lceil n/2 \rceil + 1 : n]))$$

Hogy elvarrjuk a rekurzió alját, legyen $\text{MSORT}(A[i, i])$ az üres utasítás.



Összehasonlítások száma

Jelöljük $T(n)$ -nel a lépésszámot n hosszú tömb rendezésekor. Az egyszerűség kedvéért tegyük fel, hogy $n = 2^k$.

$$T(n) \leq n - 1 + 2 T(n/2)$$

$$T(n) \leq n - 1 + 2(n/2 - 1 + 2 T(n/4)) = n - 1 + 2(n/2 - 1) + 4 T(n/4)$$

$$\vdots$$

$$T(n) \leq n - 1 + 2(n/2 - 1) + 4(n/4 - 1) + \dots + 2^{k-1}(n/2^{k-1} - 1) \leq n \lceil \log_2 n \rceil$$

felhasználva, hogy $T(1) = 0$.

Következmény

Az összefésüléssel rendezés **konstans szorzó erejéig optimális** (hiszen az alsó korlát $\approx n \log_2 n$).

- ▶ Mozgatások száma: $2n \lceil \log_2 n \rceil$
- ▶ Tárigény: $2n$ cella (bonyolultabban megcsinálva elég $n + \text{konst.}$)
- ▶ Konzervatív (stabil), és láncolt listára is jól működik.

Példa összefésüléses rendezésre

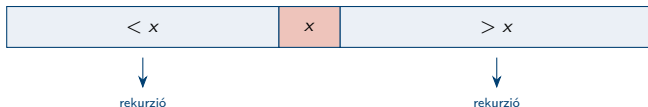
szétvágás egyeleműekre, majd összefésülés														
2		3		8		2		7		4		5		1
2 3				2 8				4 7				1 5		
2 2 3 8								1 4 5 7						
1 2 2 3 4 5 7 8														

- ▶ A rekurzió mélysége $\lceil \log_2 n \rceil$, minden szinten összesen $\mathcal{O}(n)$ munka $\Rightarrow \mathcal{O}(n \log n)$.
- ▶ A legrosszabb eset is $\mathcal{O}(n \log n)$ — ez az erőssége.

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések
- 4 Alsó becslés összehasonlítás alapú rendezésre
- 5 Összefésüléses rendezés
- 6 Gyorsrendezés**
- 7 Kulcsmanipulációs rendezések
- 8 Összefoglalás

Alapötlet — oszd meg és uralkodj, fordítva

Válasszunk egy x **pivot** elemet. Osszuk két részre a tömböt: balra az x -nél kisebbek, jobbra a nagyobbak. Ezután a két részt **külön-külön** rendezzük rekurzívan — összefésülésre már nincs szükség.



- ▶ A szétoztás (partícionálás) $n - 1$ összehasonlítás, **helyben** elvégezhető.
- ▶ A pivot a szétoztás után már **végleg a helyén** van.

Legjobb (kiegyensúlyozott) eset: a pivot mindig középre esik,

$$T(n) = 2T(n/2) + n - 1 \implies T(n) = \mathcal{O}(n \log n).$$

Legrosszabb eset: a pivot mindig a legkisebb (pl. már rendezett tömbön, ha az első elemet választjuk),

$$T(n) = T(n-1) + n - 1 \implies T(n) = \frac{n(n-1)}{2} = \mathcal{O}(n^2).$$

Átlagos eset

Véletlen (egyenletes eloszlású) input, illetve véletlenül választott pivot esetén a várható lépésszám

$$T(n) \approx 2n \ln n \approx 1.39 n \log_2 n = \mathcal{O}(n \log n).$$

- ▶ Gyakorlatban a leggyorsabb általános rendezés (kicsi konstans, jó gyorsítótár-viselkedés), de **nem konzervatív**.
- ▶ Óvintézkedések: véletlen pivot, három elem mediánja, kis résztömbökre beszűrásos rendezés.

- 1 Rendezési reláció
- 2 Keresés
- 3 A rendezési feladat, elemi rendezések
- 4 Alsó becslés összehasonlítás alapú rendezésre
- 5 Összefésüléses rendezés
- 6 Gyorsrendezés
- 7 Kulcsmanipulációs rendezések**
- 8 Összefoglalás

Alapgondolat

Nem csak összehasonlításokat használ! Pl. ismerjük az elemek számát, belső szerkezetét. $\implies$ Az $\Omega(n \log n)$ alsó korlát **nem vonatkozik rájuk**.

Ládarendezés (binsort)

Tudjuk, hogy $A[1 : n]$ elemei egy m elemű U halmazból kerülnek ki, pl. $\in \{1, \dots, m\}$.

- ▶ Lefoglalunk egy U elemeivel indexelt B tömböt (m db **ládát**), először mind üres.
- ▶ **Első fázis:** végigolvassuk A -t, és az $s = A[i]$ elemet a $B[s]$ lista **végére** fűzzük.

Mivel mindig a lista végére fűzünk, ez **konzervatív rendezés**, azaz az egyenlő elemek sorrendjét megtartja.

Tegyük fel, hogy a rendezendő $A[1 : 7]$ tömb elemei 0 és 9 közötti egészek:

A: 5 1 3 5 9 6 6

B:

0	1	2	3	4	5	6	7	8	9
	1		3		5 5	6 6			9

Második fázis: elejétől a végéig növe sorrendben végigmegyünk B -n, és a $B[i]$ listák tartalmát visszaírjuk A -ba:

A: 1 3 5 5 6 6 9

Lépésszám

B létrehozása $\mathcal{O}(m)$, első fázis $\mathcal{O}(n)$, második fázis $\mathcal{O}(n + m) \implies$ összesen $\mathcal{O}(n + m)$.

Ez gyorsabb, mint az általános alsó korlát, ha pl. $m \leq cn$.

A kulcsok **összetettek**, több komponensből állnak: $t_1 \dots t_k$ alakú szavak, ahol a t_i komponens az L_i rendezett típusból való, legyen $|L_i| = s_i$. A rendezés **lexikografikus**.

Példa: dátumok

Legyen $(U, <)$ a huszadik századi dátumok összessége az időrendnek megfelelő rendezéssel.

$$\begin{array}{ll} L_1 = \{1900, 1901, \dots, 1999\}, & s_1 = 100 \\ L_2 = \{\text{január}, \text{február}, \dots, \text{december}\}, & s_2 = 12 \\ L_3 = \{1, 2, \dots, 31\}, & s_3 = 31 \end{array}$$

A dátumok rendezése éppen az L_i típusokból származó lexikografikus rendezés lesz.

Az eljárás

- ① Rendezzük a sorozatot az **utolsó**, a k -edik komponens szerint ládarendezéssel.
- ② A kapottat rendezzük a $k - 1$ -edik komponens szerint ládarendezéssel.
- ③ ...és így tovább, végül az **első** komponens szerint.

- ▶ Fontos, hogy a ládarendezésnél az elemeket a ládában mindig a lista **végére** tettük.
- ▶ Így ha két azonos kulcsú elem közül az egyik megelőzi a másikat, akkor a rendezés után sem változik a sorrendjük.
- ▶ → Az ilyen rendezést **konzervatív rendezésnek** nevezzük.

Meglepő, de a rendezést a *legkevésbé* fontos komponenssel kell kezdeni!

Miért működik a radix jól?

Ha $X < Y$, az első $i - 1$ tag megegyezik, de $x_i < y_i$, akkor az i -edik komponens rendezésekor X előre kerül. A ládarendezés konzervatív $\implies$ később már nem változik a sorrendjük.

kiindulás	napok szerint	hónapok szerint	évek szerint
1969.01.18.	1969.01.01.	1969.01.01.	1918.12.18.
1955.12.18.	1955.01.18.	1955.01.18.	1955.01.18.
1918.12.18.	1955.12.18.	1969.01.18.	1955.12.18.
1955.01.18.	1918.12.18.	1955.12.18.	1969.01.01.
1969.01.01.	1969.01.18.	1918.12.18.	1969.01.18.

A harmadik menet végén a sorozat **teljesen rendezett**.

A k ládarendezés összköltsége:

$$\mathcal{O}\left(kn + \sum_{i=1}^k s_i\right)$$

Ez lehet gyorsabb az általános korlátnál

Ha c, k állandók és $s_i \leq cn$, akkor

$$\mathcal{O}\left(kn + \sum_{i=1}^k cn\right) = \mathcal{O}(k(c+1)n) = \mathcal{O}(n).$$

Példa

Az $[1, n^{10} - 1]$ intervallumból való egészek rendezése kettes (vagy n alapú) számrendszerbeli jegyek szerint: $k = \log n$, $s_i = 2 \implies \mathcal{O}(n \log n + 2 \log n) = \mathcal{O}(n \log n)$.

Tehát a radix csak akkor nyer, ha a komponensek száma kicsi és a típusok mérete korlátos.

Feladat	Módszer	Összehasonlítások (legrosszabb eset)
keresés rendezetlenben	végignézés	n
keresés rendezettben	lineáris keresés	n
keresés rendezettben	bináris keresés	$\lceil \log_2(n+1) \rceil$ (optimális)
minimumkeresés	kiesés verseny	$n-1$ (optimális)

- ▶ Mindkét optimalitást **ellenség-stratégiával**, illetve gráfos érveléssel bizonyítottuk.
- ▶ A rendezettség tehát a keresést n -ről $\log n$ -re gyorsítja — ezért éri meg rendezni.

Módszer	legrosszabb	átlagos	tár	konzervatív	elv
buborék	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	igen	összehasonlítás
beszúrásos	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	igen	összehasonlítás
összefésüléses	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	igen	oszd meg és uralkodj
gyorsrendezés	$\mathcal{O}(n^2)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(\log n)$	nem	oszd meg és uralkodj
láda	$\mathcal{O}(n + m)$	$\mathcal{O}(n + m)$	$\mathcal{O}(m)$	igen	kulcsmanipuláció
radix	$\mathcal{O}(kn + \sum s_i)$	ua.	$\mathcal{O}(n + \max s_i)$	igen	kulcsmanipuláció

- ▶ Összehasonlítás alapon $(n \log n)$ nagyságrend alá **nem** lehet menni — ezt az összefésüléses rendezés eléri.
- ▶ A kulcsok szerkezetének ismeretében ez a korlát **megkerülhető** (láda-, radix rendezés).

- [1] Katona Gyula Y.: *Algoritmuselmélet* — 5. előadás: Keresés, rendezés. BME SZIT, 2012.
- [2] Katona Gyula Y., Recski András, Szabó Csaba: *A számítástudomány alapjai*. Typotex, Budapest.
- [3] Cormen, Leiserson, Rivest, Stein: *Új algoritmusok* (Introduction to Algorithms). Scolar Kiadó.
- [4] D. E. Knuth: *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley.